X-CAVATE

CLI & Python API Reference

Version 2.0.0

Convert vascular network geometries into collision-free
3D printer toolhead pathways and G-code.

SM = Single-Material | **MM** = Multimaterial

This document combines content from the original X-CAVATE Supplement User Manual with new CLI and Python API documentation for the refactored package. Sections containing new text are marked with a purple **"Text added by Claude"** callout box.

March 10, 2026

Contents

1 Introduction **3**

2 Setup **3**
- 2.1 Original Setup Instructions . . . 3
- 2.2 Installation . . . 3
  - 2.2.1 Option A: Conda (Recommended) . . . 3
  - 2.2.2 Option B: pip . . . 4
  - 2.2.3 Option C: Docker . . . 4

3 Input Files **4**
- 3.1 Inputs Overview . . . 4
  - 3.1.1 Extension Folder . . . 4
  - 3.1.2 Custom G-code Folder . . . 5
- 3.2 Network File Format . . . 5
- 3.3 Inlet / Outlet File Format . . . 5

4 Running X-CAVATE via CLI **6**
- 4.1 Basic Invocation . . . 6
- 4.2 Complete CLI Argument Reference . . . 6
  - 4.2.1 Required Arguments . . . 6
  - 4.2.2 Geometry Parameters . . . 7
  - 4.2.3 Print Settings . . . 7
  - 4.2.4 Multimaterial Settings . . . 8
  - 4.2.5 Gap Closure . . . 8
  - 4.2.6 Positive Ink Displacement . . . 8
  - 4.2.7 Advanced Flags . . . 8
- 4.3 Example Commands . . . 9
  - 4.3.1 Minimal Single-Material Run . . . 9
  - 4.3.2 Multimaterial with Speed Calculation and Custom G-code . . . 9
  - 4.3.3 Using a Custom Output Directory . . . 9

5 Running X-CAVATE via Python API **9**
- 5.1 Quick Start . . . 10
- 5.2 XcavateConfig Dataclass . . . 10
- 5.3 Enums . . . 12
- 5.4 run_xcavate() Function . . . 12
- 5.5 Complete Example . . . 13

6 Calibration **14**
- 6.1 Updating Vessel Radii . . . 14
- 6.2 Q Computation Equations . . . 14
- 6.3 Python API: Calibration Functions . . . 15
  - 6.3.1 compute_flow_rate() . . . 15
  - 6.3.2 validate_calibration() . . . 15

7 Outputs **16**

7.1 Output Overview 16
7.2 Directory Structure 16
7.3 Output File Table 17
7.4 Changelog 17
7.5 Plots 18

8 Gap Closure / Segment Extension 18
8.1 Identifying Print Passes for Segment Extension 18

9 X-CAVATE Structure: Description of Network Processing 18
9.1 Pre-processing 18
9.2 Modified Depth-First Search (DFS) 19
9.3 Post-processing 20
9.4 Radius Computation 21
9.5 Generation of Print Passes for Multimaterial Printing 21

10 Module Structure 21

1 Introduction

Text added by Claude

X-CAVATE (Cross-Collision Avoidance for VAscular Tree Embedding) converts vascular network geometries exported from SimVascular into collision-free 3D printer toolhead pathways and G-code. The software supports three usage modes:

1. **Streamlit GUI** — A web-based graphical interface requiring no coding experience. See the companion *GUI User Guide* for full documentation.
2. **Command-Line Interface (CLI)** — The `xcavate` command for scripted and batch workflows. Documented in Section 4.
3. **Python API** — Import `xcavate` as a library for programmatic access. Documented in Section 5.

This document serves as the reference manual for the CLI and Python API. It also preserves the theoretical background and workflow descriptions from the original supplement user manual.

2 Setup

2.1 Original Setup Instructions

Navigate to https://github.com/sohams-MASS/X-CAVATE. Click on the green "Code" button and download the `.zip` file. Place the downloaded folder in the directory of your choice, such as the Desktop.

[See original supplement for screenshot.]

Text added by Claude

The repository URL has changed from the original `github.com/jessica-herrmann/xcavate` to the current `github.com/sohams-MASS/X-CAVATE`. The folder structure has been refactored from a single-script layout into a proper Python package.

2.2 Installation

2.2.1 Option A: Conda (Recommended)

1. Open a terminal (Mac: Terminal.app; Windows: Anaconda Prompt or PowerShell).
2. Confirm conda is installed:

```
conda --version
```

   If not installed, download Miniconda from https://docs.conda.io/en/latest/miniconda.html.

3. Clone the repository and enter the directory:

```
git clone https://github.com/sohams-MASS/X-CAVATE.git
cd X-CAVATE
```

4. Create and activate the conda environment:

```
conda env create -f environment.yml
conda activate xcavate-gui
```

5. Verify the installation:

```
xcavate --help
```

2.2.2 Option B: pip

1. Confirm Python 3.9+ is installed: `python3 --version`
2. Clone the repository (same as above).
3. Install with GUI support:

```
pip install ".[gui]"
```

For CLI-only usage (no web interface): `pip install .`

2.2.3 Option C: Docker

1. Install Docker Desktop from https://www.docker.com/products/docker-desktop.
2. Build the image:

```
docker build -t xcavate .
```

3. Run the container:

```
docker run -p 8501:8501 xcavate
```

4. Open http://localhost:8501 in your browser.

3 Input Files

3.1 Inputs Overview

The *inputs* folder contains two subfolders: *custom* and *extension*.

[See original supplement for screenshot.]

3.1.1 Extension Folder

For a description of how to utilize the *inputs/extension* folder to implement segment extension (gap closure), see Section 8 of this guide.

3.1.2 Custom G-code Folder

The *inputs/custom* folder contains sample `.txt` files which can be updated to contain custom G-code wrappers to adapt X-CAVATE to your specific printer. For example, to provide your own G-code for header information, replace the placeholder text within `header_code.txt` with your own code to accomplish this task.

[See original supplement for screenshot.]

The screenshot below of an output file, `gcode_SM_positiveInk.txt`, provides an example of integrating custom G-code into the output G-code `.txt` file generated for single material printing with a positive ink displacement-based printer. Editing the `header_code.txt` file in advance of running X-CAVATE will replace the placeholder text in the first line with the custom G-code saved within `header_code.txt`.

[See original supplement for screenshot.]

Place your two `.txt` output files from SimVascular, one containing the network splines as coordinate lists, and the other containing the identities of the inlet and outlet nodes, into the *inputs* folder.

[See original supplement for screenshot.]

3.2 Network File Format

```
Vessel: 0, Number of Points: 100
33.4878, 0.0000, 53.5303, 0.0200
33.4115, -0.5285, 52.7311, 0.0200
...
Vessel: 1, Number of Points: 100
15.0600, -37.3300, 11.6600, 0.0190
...
```

Column	Required	Description
1–3	Yes	x, y, z coordinates (cm)
4	No	Vessel radius (needed for speed calculation)
5	No	Arterial/venous flag: 0 = venous, nonzero = arterial (needed for multi-material)

Warning

Coordinates in the network file must be in **centimeters** — X-CAVATE converts to millimeters internally (multiplication by 10).

3.3 Inlet / Outlet File Format

```
inlet
21.97, -15.43, 50.00
11.81, 0.00, 21.34
outlet
36.05, -27.19, 50.00
50.00, 0.00, 26.54
```

Coordinates are matched to the nearest network point automatically.

4 Running X-CAVATE via CLI

Text added by Claude

The original workflow required typing `python3 xcavate.py` followed by all parameters, and optionally using the `command_line_writer.py` helper script to generate the command string. In the refactored package, X-CAVATE is installed as a console entry point. The command is simply `xcavate`:

```
xcavate --network_file <path> --inletoutlet_file <path> [OPTIONS]
```

The `command_line_writer.py` script is no longer needed — all parameters are passed directly as CLI flags.

4.1 Basic Invocation

```
xcavate \
  --network_file inputs/network.txt \
  --inletoutlet_file inputs/inlet_outlet.txt \
  --multimaterial 0 \
  --tolerance_flag 0 \
  --nozzle_diameter 0.5 \
  --container_height 10 \
  --num_decimals 3 \
  --speed_calc 0 \
  --plots 1 \
  --downsample 0 \
  --custom 0 \
  --printer_type 0
```

4.2 Complete CLI Argument Reference

Text added by Claude

The following tables document every CLI argument. They are derived directly from the argument parser in `xcavate/cli.py`.

4.2.1 Required Arguments

Flag	Type	Default	Description
`--network_file`	str	—	Path to `.txt` file containing network coordinates.
`--inletoutlet_file`	str	—	Path to `.txt` file containing inlet and outlet coordinates.
`--multimaterial`	int	—	Multimaterial mode (1=yes, 0=no).

Flag	Type	Default	Description
`--tolerance_flag`	float	0	Include tolerance? (1=yes, 0=no).
`--tolerance`	float	0	Amount of tolerance (0 is none).
`--nozzle_diameter`	float	—	Outer diameter of the nozzle (mm).
`--container_height`	float	—	Height of the print container (mm).
`--num_decimals`	int	—	Number of decimal places for rounding output values.
`--speed_calc`	int	—	Compute print speeds for changing radii? (1=yes, 0=no).
`--plots`	int	—	Generate plots? (1=yes, 0=no).
`--downsample`	int	—	Downsample network? (1=yes, 0=no).
`--custom`	int	—	Providing custom G-code? (1=yes, 0=no).
`--printer_type`	int	0	Type of printer: 0=Pressure, 1=Positive Ink, 2=Aerotech.
`--amount_up`	float	10	Z-raise above container for nozzle switches (mm).

4.2.2 Geometry Parameters

Flag	Type	Default	Description
`--container_x`	float	50	Container x-dimension (mm).
`--container_y`	float	50	Container y-dimension (mm).
`--scale_factor`	float	1	Network scale factor.
`--top_padding`	float	0	Padding above network (mm).
`--downsample_factor`	int	1	Downsample factor.

4.2.3 Print Settings

Flag	Type	Default	Description
`--flow`	float	0.161	Volumetric flow rate (mm^3/s).
`--print_speed`	float	1	Print speed (mm/s).
`--jog_speed`	float	5	Jog speed (mm/s).
`--jog_speed_lift`	float	0.25	Z-lift jog speed (mm/s).
`--initial_lift`	float	0.5	Initial lift distance (mm).
`--jog_translation`	float	10	Jog speed between nozzles (mm/s).
`--dwell_start`	float	0.08	Dwell at start (s). Aerotech only.
`--dwell_end`	float	0.08	Dwell at end (s). Aerotech only.

4.2.4 Multimaterial Settings

Flag	Type	Default	Description
`--offset_x`	float	103	Printhead x-offset (mm).
`--offset_y`	float	0.5	Printhead y-offset (mm).
`--front_nozzle`	int	1	1 if venous nozzle in front, 2 if behind.
`--printhead_1`	str	Aa	Arterial printhead name.
`--printhead_2`	str	Ab	Venous printhead name.
`--axis_1`	str	A	Arterial z-axis name.
`--axis_2`	str	B	Venous z-axis name.
`--resting_pressure`	float	0	Resting pressure (psi).
`--active_pressure`	float	5	Active pressure (psi).

4.2.5 Gap Closure

Flag	Type	Default	Description
`--num_overlap`	int	0	Overlap nodes at pass boundaries.
`--overlap_algorithm`	str	retrace	Overlap algorithm: `retrace` (original) or `consecutive` (fast).
`--close_sm`	int	0	Load gap closure file for SM? (1=yes, 0=no).
`--close_mm`	int	0	Load gap closure file for MM? (1=yes, 0=no).

4.2.6 Positive Ink Displacement

Flag	Type	Default	Description
`--positiveInk_start`	float	0	Plunger displacement at pass start.
`--positiveInk_end`	float	0	Plunger displacement at pass end.
`--positiveInk_radii`	int	0	Use per-node vessel radii? (1=yes, 0=no).
`--positiveInk_diam`	float	1	Fixed vessel diameter (mm).
`--positiveInk_syringe_diam`	float	1	Syringe barrel diameter (mm).
`--positiveInk_factor`	float	1	Extrusion scaling factor.
`--positiveInk_start_arterial`	float	0	Arterial plunger displacement at start.
`--positiveInk_start_venous`	float	0	Venous plunger displacement at start.
`--positiveInk_end_arterial`	float	0	Arterial plunger displacement at end.
`--positiveInk_end_venous`	float	0	Venous plunger displacement at end.

4.2.7 Advanced Flags

Text added by Claude

These flags were added in the refactored version and were not present in the original `xcavate.py` script.

Flag	Type	Default	Description
`--algorithm`	str	dfs	Pathfinding algorithm: `dfs` (depth-first search).
`--output_dir`	str	outputs	Output directory path.

4.3 Example Commands

Text added by Claude

The examples below are adapted from the original supplement user manual, updated for the new `xcavate` CLI entry point.

4.3.1 Minimal Single-Material Run

```
xcavate \
  --network_file inputs/network.txt \
  --inletoutlet_file inputs/inlet_outlet.txt \
  --multimaterial 0 --tolerance_flag 0 \
  --nozzle_diameter 0.65 --container_height 50 \
  --num_decimals 5 --amount_up 10 \
  --speed_calc 0 --plots 1 --downsample 0 \
  --custom 0 --printer_type 0
```

4.3.2 Multimaterial with Speed Calculation and Custom G-code

```
xcavate \
  --network_file inputs/multimaterial_network.txt \
  --inletoutlet_file inputs/inlet_outlet_multimaterial.txt \
  --multimaterial 1 --tolerance_flag 0 --tolerance 0 \
  --nozzle_diameter 0.65 --num_decimals 5 \
  --amount_up 10 --container_height 50 \
  --speed_calc 1 --plots 1 --downsample 0 \
  --flow 0.127 --custom 1 --printer_type 0
```

4.3.3 Using a Custom Output Directory

```
xcavate \
  --network_file inputs/network.txt \
  --inletoutlet_file inputs/inlet_outlet.txt \
  --multimaterial 0 --tolerance_flag 0 \
  --nozzle_diameter 0.5 --container_height 10 \
  --num_decimals 3 --speed_calc 0 --plots 1 \
  --downsample 0 --custom 0 --printer_type 0 \
  --output_dir my_results
```

5 Running X-CAVATE via Python API

Text added by Claude

This entire section documents the Python API introduced in the refactored package. It enables programmatic access to X-CAVATE for scripting, batch processing, and integration into larger workflows.

5.1 Quick Start

```python
from pathlib import Path
from xcavate.config import XcavateConfig
from xcavate.pipeline import run_xcavate

config = XcavateConfig(
    network_file=Path("inputs/network.txt"),
    inletoutlet_file=Path("inputs/inlet_outlet.txt"),
    nozzle_diameter=0.5,
    container_height=10.0,
    num_decimals=3,
)

results = run_xcavate(config)

# Access results
print(f"SM passes: {len(results['print_passes_sm'])}")
print(f"Output in: {config.output_dir}/")
```

5.2 XcavateConfig Dataclass

The `XcavateConfig` dataclass (defined in `xcavate/config.py`) replaces the 30+ global variables from the original script. All fields with defaults are optional.

Field	Type	Default	Description
Required parameters			
`network_file`	Path	—	Path to network coordinates file.
`inletoutlet_file`	Path	—	Path to inlet/outlet file.
`nozzle_diameter`	float	—	Outer diameter of nozzle (mm).
`container_height`	float	—	Height of print container (mm).
`num_decimals`	int	—	Decimal places for output rounding.
Feature flags			
`amount_up`	float	10.0	Z-raise between passes (mm).
`multimaterial`	bool	False	Enable multimaterial mode.
`tolerance_flag`	bool	False	Enable tolerance.
`speed_calc`	bool	False	Compute per-node speeds from radii.
`generate_plots`	bool	True	Create 3D HTML visualizations.
`downsample`	bool	False	Enable downsampling.
`custom_gcode`	bool	False	Use custom G-code templates.
`printer_type`	PrinterType	PRESSURE	Printer type enum.

Field	Type	Default	Description
`algorithm`	PathfindingAlgorithm	DFS	Pathfinding strategy enum.
Geometry			
`tolerance`	float	0.0	Tolerance distance (mm).
`scale_factor`	float	1.0	Network scale multiplier.
`top_padding`	float	0.0	Padding above network top (mm).
`container_x`	float	50.0	Container width (mm).
`container_y`	float	50.0	Container depth (mm).
`convert_factor`	float	10.0	Unit conversion (cm → mm).
`downsample_factor`	int	1	Downsample factor.
Print settings			
`flow`	float	0.161	Volumetric flow rate (mm^3/s).
`print_speed`	float	1.0	Print speed (mm/s).
`jog_speed`	float	5.0	Jog speed (mm/s).
`jog_speed_lift`	float	0.25	Z-lift jog speed (mm/s).
`jog_translation`	float	10.0	Inter-nozzle jog speed (mm/s).
`initial_lift`	float	0.5	Initial lift distance (mm).
`dwell_start`	float	0.08	Dwell at start (s).
`dwell_end`	float	0.08	Dwell at end (s).
Multimaterial			
`offset_x`	float	103.0	Printhead x-offset (mm).
`offset_y`	float	0.5	Printhead y-offset (mm).
`front_nozzle`	int	1	1=venous in front, 2=behind.
`printhead_1`	str	"Aa"	Arterial printhead name.
`printhead_2`	str	"Ab"	Venous printhead name.
`axis_1`	str	"A"	Arterial z-axis name.
`axis_2`	str	"B"	Venous z-axis name.
`resting_pressure`	float	0.0	Resting pressure (psi).
`active_pressure`	float	5.0	Active pressure (psi).
Gap closure			
`num_overlap`	int	0	Overlap nodes at boundaries.
`overlap_algorithm`	OverlapAlgorithm	RETRACE	Overlap algorithm enum.
`close_sm`	bool	False	Load SM gap closure file.
`close_mm`	bool	False	Load MM gap closure file.
Positive ink displacement			
`positive_ink_start`	float	0.0	Plunger displacement at pass start.
`positive_ink_end`	float	0.0	Plunger displacement at pass end.
`positive_ink_radii`	bool	False	Use per-node vessel radii.
`positive_ink_diam`	float	1.0	Fixed vessel diameter (mm).

Field	Type	Default	Description
positive_ink_syringe_diam	float	1.0	Syringe barrel diameter (mm).
positive_ink_factor	float	1.0	Extrusion scaling factor.
positive_ink_start_arterial	float	0.0	Arterial start displacement.
positive_ink_start_venous	float	0.0	Venous start displacement.
positive_ink_end_arterial	float	0.0	Arterial end displacement.
positive_ink_end_venous	float	0.0	Venous end displacement.
Paths			
custom_gcode_dir	Path	inputs/custom	Directory for custom G-code templates.
extension_dir	Path	inputs/extension	Directory for gap extension files.
output_dir	Path	outputs	Output directory.

5.3 Enums

```
from xcavate.config import PrinterType, PathfindingAlgorithm, OverlapAlgorithm

# PrinterType
PrinterType.PRESSURE       # 0 - Pneumatic extrusion
PrinterType.POSITIVE_INK   # 1 - Positive ink displacement
PrinterType.AEROTECH       # 2 - Aerotech 6-axis controller

# PathfindingAlgorithm
PathfindingAlgorithm.DFS        # "dfs" - Modified depth-first search
# Only DFS is currently supported

# OverlapAlgorithm
OverlapAlgorithm.RETRACE      # "retrace" - Scan all previous passes
OverlapAlgorithm.CONSECUTIVE   # "consecutive" - Check adjacent pairs
```

5.4 run_xcavate() Function

```
def run_xcavate(
    config: XcavateConfig,
    progress_cb: Optional[Callable[[int, str], None]] = None,
) -> dict:
```

Parameters:

config Fully populated `XcavateConfig` instance.

progress_cb Optional callback `(step_number, description)` for progress reporting. Steps are numbered 1–7.

Returns a dict with the following keys:

Key	Description
`print_passes_sm`	Single-material print passes (dict of pass index → node list).
`print_passes_mm`	Multimaterial print passes (or `None`).
`points`	Interpolated coordinate array (numpy ndarray).
`points_original`	Original coordinate array before interpolation.
`coord_num_dict_original`	Original vessel → point count mapping.
`changelog`	List of changelog lines from gap closure.
`speed_map_sm`	Speed map for SM (or `None`).
`speed_map_mm`	Speed map for MM (or `None`).
`material_map`	Material classification per pass (or `None`).
`instructions`	Operator instructions dict (start position, padding, etc.).

5.5 Complete Example

```python
from pathlib import Path
from xcavate.config import (
    XcavateConfig,
    PrinterType,
    PathfindingAlgorithm,
)
from xcavate.pipeline import run_xcavate

config = XcavateConfig(
    network_file=Path("inputs/multimaterial_network.txt"),
    inletoutlet_file=Path("inputs/inlet_outlet_mm.txt"),
    nozzle_diameter=0.65,
    container_height=50.0,
    num_decimals=5,
    multimaterial=True,
    speed_calc=True,
    flow=0.127,
    generate_plots=True,
    custom_gcode=True,
    printer_type=PrinterType.PRESSURE,
    algorithm=PathfindingAlgorithm.DFS,
    output_dir=Path("my_output"),
)

results = run_xcavate(config)

# Inspect results
sm = results["print_passes_sm"]
mm = results["print_passes_mm"]
print(f"Generated {len(sm)} SM passes")
if mm:
    print(f"Generated {len(mm)} MM passes")

# Access start position for operator instructions
instr = results["instructions"]
pos = instr["start_position"]
print(f"Start at X={pos['x']}, Y={pos['y']}, Z={pos['z']}")
```

6 Calibration

6.1 Updating Vessel Radii

Follow the calibration protocol to create `diameters.csv` and `diameters_validation.csv`. In the original workflow, these files were placed within the *xcavate-main/inputs* folder and processed by running `python3 calibration.py` from the command line. The computed volumetric flow rate, Q, was output to the command line interface and saved within a subfolder *xcavate-main/outputs/radii*.

[See original supplement for screenshots.]

The calibration curve appears within `radii_pressure.html`.

[See original supplement for screenshot.]

The percent error outputs to the command line interface, as well as saves in tabular format to the output folder as `error_pressure.html`.

[See original supplement for screenshots.]

Text added by Claude

In the refactored version, calibration is performed through either:

- The **Calibration** tab in the Streamlit GUI (see GUI User Guide).
- The Python API functions described below.

The standalone `calibration.py` and `calibration_validation.py` scripts have been replaced by the `xcavate.core.calibration` module.

6.2 Q Computation Equations

To incorporate the specified vessel radii, X-CAVATE continuously updates the print speed of each coordinate within the network, at constant pressure, to dynamically alter the printed filament diameter. Computation of the print speed depends on the following definition of volumetric flow rate:

Equation 1:

$$Q = \frac{V}{t} \tag{1}$$

where Q is the volumetric flow rate of the ink through the nozzle orifice, V is the volume of the printed filament, and t is the duration of the print. Assuming that the cylindrical nozzle extrudes a cylindrical filament, V becomes $\pi r^2 L$, where r is the filament radius and L is the length of the printed filament. Substituting into Equation 1 results in:

Equation 2:

$$Q = \frac{\pi r^2 L}{t} \tag{2}$$

Since L is equivalent to the nozzle translation velocity, v, multiplied by the duration of travel, t, Equation 2 becomes:

Equation 3:

$$Q = \frac{\pi r^2 v t}{t} \tag{3}$$

Rearrangement of Equation 3 results in:

Equation 4:

$$r = \sqrt{\frac{Q}{\pi v}} \tag{4}$$

Thus, for a known filament radius r and ink volumetric flow rate Q, the nozzle translation velocity v can be calculated.

6.3 Python API: Calibration Functions

Text added by Claude

The calibration functions below are defined in `xcavate/core/calibration.py` and can be called directly from Python.

6.3.1 compute_flow_rate()

```
from xcavate.core.calibration import compute_flow_rate

result = compute_flow_rate(
    measured_csv_bytes=open("diameters.csv", "rb").read(),
    print_speeds=[0.5, 1.0, 2.0],      # mm/s, one per group
    measurements_per_target=3,           # rows to average per group
    output_dir=Path("outputs/radii"),    # optional: save to disk
)

print(f"Q = {result['q_value']:.4f} mm^3/s")
# result also contains: 'figure', 'q_txt_bytes', 'html_bytes'
```

Parameters:

`measured_csv_bytes` Raw bytes of a CSV file with a `Length` column containing measured filament diameters in **microns**.

`print_speeds` Print speeds in mm/s used during calibration, one per group. Pressure is held constant; speed is varied.

`measurements_per_target` Number of rows in the CSV to average for each print speed group (default: 3).

`output_dir` If provided, `Q_pressure.txt` and `radii_pressure.html` are written to this directory.

Returns a dict with keys: `q_value` (float), `figure` (Plotly Figure), `q_txt_bytes` (bytes), `html_bytes` (bytes).

6.3.2 validate_calibration()

```
from xcavate.core.calibration import validate_calibration

result = validate_calibration(
    target_diameters=[0.5, 0.8, 1.0],    # expected diameters in mm
    measured_csv_bytes=open("diameters_validation.csv", "rb").read(),
```

```python
    measurements_per_target=3,
)

for t, m, e in zip(
    result["target"], result["measured_avg"], result["percent_error"]
):
    print(f"Target: {t} mm, Measured: {m} mm, Error: {e}%")
# result also contains: 'csv_bytes', 'html_bytes', 'figure'
```

Parameters:

`target_diameters` Expected diameters in mm.

`measured_csv_bytes` Raw bytes of a CSV file with a `Length` column containing measured diameters in **microns**.

`measurements_per_target` Number of rows to average per target diameter (default: 3).

Returns a dict with keys: `target`, `measured_avg`, `percent_error` (lists), `csv_bytes` (bytes), `html_bytes` (bytes), `figure` (Plotly Figure).

7 Outputs

7.1 Output Overview

The output from X-CAVATE automatically saves within the *outputs* folder (or the directory specified by `--output_dir`), which, prior to running X-CAVATE, is empty. Output takes the form of G-code files, network plots, the mathematical graph describing the network, and a changelog. The plots depict the individual print passes (if the plots parameter was enabled), and the *graph* folder contains individual files breaking down the network into various formats, including nodes, edges, and coordinates, for ease of future data manipulation.

[See original supplement for screenshot.]

7.2 Directory Structure

```
outputs/
  graph/
    graph.txt               # Adjacency graph
    special_nodes.txt       # Branchpoints, endpoints, inlets, outlets
    SM_x_coords.txt         # Single-material x coordinates per pass
    SM_y_coords.txt         # Single-material y coordinates per pass
    SM_z_coords.txt         # Single-material z coordinates per pass
    SM_combined.txt         # Combined xyz per pass
    MM_x_coords.txt         # Multimaterial x coordinates (if enabled)
    MM_y_coords.txt         # Multimaterial y coordinates (if enabled)
    MM_z_coords.txt         # Multimaterial z coordinates (if enabled)
    MM_combined.txt         # Multimaterial combined xyz (if enabled)
  gcode/
    gcode_SM_pressure.txt   # Single-material G-code
    gcode_MM_pressure.txt   # Multimaterial G-code (if enabled)
  plots/
    network_original.html   # 3D plot of original network
```

```
    network_SM.html          # 3D plot of SM print passes
    network_MM.html          # 3D plot of MM print passes (if enabled)
  changelog.txt              # Gap closure operations log
```

Info

The G-code filename suffix matches your selected printer type: `pressure`, `positive_ink`, or `aerotech`.

7.3 Output File Table

File	Directory	Description
`graph.txt`	`graph/`	List of nodes and edges constituting the mathematical graph describing the network.
`special_nodes.txt`	`graph/`	Node numbers corresponding to "special nodes" within the graph, i.e. branchpoints, endpoints, inlets, outlets, and daughter nodes.
`SM_x_coords.txt`	`graph/`	Cartesian x-coordinates corresponding to nodes within the network, grouped by print pass and print order (for single material printing).
`SM_y_coords.txt`	`graph/`	Cartesian y-coordinates, grouped by print pass and print order (for single material printing).
`SM_z_coords.txt`	`graph/`	Cartesian z-coordinates, grouped by print pass and print order (for single material printing).
`SM_combined.txt`	`graph/`	List of Cartesian coordinates describing the network, grouped by individual print pass for the passes constituting the single material print.
`MM_*.txt`	`graph/`	Multimaterial equivalents of the above (if enabled).
`gcode_SM_*.txt`	`gcode/`	Single-material G-code for the selected printer type.
`gcode_MM_*.txt`	`gcode/`	Multimaterial G-code (if enabled).
`network_original.html`	`plots/`	Interactive 3D visualization of the original vascular network.
`network_SM.html`	`plots/`	Interactive 3D visualization of SM print passes.
`network_MM.html`	`plots/`	Interactive 3D visualization of MM print passes.
`changelog.txt`	root	Log of all gap closure operations performed.

7.4 Changelog

The `changelog.txt` file logs progress through X-CAVATE, allowing the user to understand exactly how the mathematical graph defining the vascular network has been created and manipulated by the program. For a description of the various pre-processing and post-processing steps involved in manipulating the network, see Section 9.

[See original supplement for screenshot.]

7.5 Plots

The *plots* folder contains visual plots of the networks, subdivided by print passes.

[See original supplement for screenshots.]

The plots enable the user to step through the print passes chronologically, visually replicating the order in which the printer will construct the network.

[See original supplement for screenshot of stepping through individual print passes.]

8 Gap Closure / Segment Extension

8.1 Identifying Print Passes for Segment Extension

To identify an individual print pass for segment extension, open the network plot after running X-CAVATE. Hover over the endpoint node of interest; an informational box will appear which contains the x, y, z coordinates of the node, along with the print path (e.g., trace 35 refers to print pass 35).

[See original supplement for screenshot.]

To create additional extensions for single material printing, replace the placeholder text within `pass_to_extend_SM.txt` and `deltas_to_extend_SM.txt`. Use the `pass_to_extend_SM.txt` file to identify the numbers of the print passes, one per line, containing the endpoints to extend. Use the `deltas_to_extend_SM.txt` to specify the linear distance by which to extend the x-, y-, and z-coordinates of the endpoint. In the example below, print pass 4 will be extended by 2.0 mm in the x-direction, 3.5 mm in the y-direction, and 1.4 mm in the z-direction. Print pass 6 will be extended by 5.2 mm in the x-direction, 1.3 mm in the y-direction, and 2.4 mm in the z-direction. The process is the same for the multimaterial (_MM) files.

[See original supplement for screenshots.]

9 X-CAVATE Structure: Description of Network Processing

9.1 Pre-processing

X-CAVATE accepts an input vascular network in the form of two text (`.txt`) files. One of these files specifies the network in terms of the Cartesian coordinates composing its vessel splines, and the other specifies the coordinates of the inlets and outlets. The first text file, named "VascularNetwork.txt," consists of a minimum of 3 columns and a maximum of 5 columns. Column 1 contains the x-coordinates, column 2 contains the y-coordinates, and column 3 contains the z-coordinates; these three columns are mandatory. Column 4, if included, contains the radii measurements. Column 5, if included, contains the assignment as either arterial (represented with a "0") or venous (represented with a "1"). Each row of VascularNetwork.txt therefore specifies an individual coordinate within one of the splines constituting the vascular network model. The coordinates are further grouped within the text file by vessel number, which is specified as a header above each sublist of coordinates. The second text file, called "InletOutlet.txt," specifies which coordinates within the network are the network inlets and outlets. The coordinates within InletOutlet.txt must match their appearance within VascularNetwork.txt exactly, meaning, for example, that if an inlet coordinate contains 10 digits within InletOutlet.txt, it must contain the same 10 digits within VascularNetwork.txt.

Upon importation of the network, X-CAVATE reformats the input file content into a graph. In a graph, coordinates termed "nodes" are connected to one another by lines, called "edges." Any two nodes which are connected to one another by one edge are termed "neighbors." Each node is also its own neighbor. Edges may have an orientation, or defined direction of traversal between its two nodes, in which case the graph is called "directed." Graphs whose edges are unoriented, or directionless, are called "undirected," and graphs consisting of a combination of oriented and unoriented edges are called "mixed" (Tarjan).

To label each coordinate with a nodal identity, the coordinates within the network are consecutively assigned whole numbers beginning with "0" until all nodes within the network have been assigned a number. To assist with parsing the network into subgroups of neighboring nodes, we then introduce specific terminology to define the types of nodes within the network. These include "endpoint" nodes, "parent branch point" nodes, "daughter" nodes, and "standard" nodes. "Endpoint" nodes are nodes which are either the first node or the last node of an individual vessel. Inlet and outlet nodes are both forms of endpoint nodes which are only connected by one edge to one node within the network. "Parent branch point" nodes are endpoint nodes which form the first node of a daughter vessel which branches off of an existing parent vessel. The parent branch point node always exists on the parent vessel from which it branches, meaning that it exists on the line formed between its two "daughter" nodes. The "daughter" nodes are members of the parent vessel and are neighbors of the parent branch point. Parent branch points therefore always have four neighbors: the two "daughter" nodes, itself, and the node to which it is connected along the daughter vessel (i.e., the second node within the daughter vessel). This latter node is a "standard" node, which is the term given to any node within the network that is not an endpoint, a parent branch point, or a daughter node. The printed ink filaments are represented by the edges within the graph.

Prior to generating the graph, X-CAVATE performs a linear interpolation to ensure that no adjacent coordinates within individual vessels are separated by a distance greater than the nozzle radius. This step introduces additional coordinates to allow the nodes to approximate the edges, which assists with collision detection during the modified depth-first search portion of the algorithm. The linear interpolation does not affect the network geometry because, in each case, it introduces the coordinates along the line which already exists between the original adjacent nodes.

9.2 Modified Depth-First Search (DFS)

Following interpolation and structuring of the graph, X-CAVATE performs a modified depth-first search (DFS). DFS is a standard algorithm for traversing all nodes within a graph. Initially, all nodes within the graph are marked as "unvisited," meaning that they have not been traversed by the algorithm. To explore the graph, DFS begins from an initial node and traverses one of its connected edges to a neighboring node. From that node, DFS selects an untraveled edge and repeats this process until eventually reaching a node whose neighbors have all been visited. When this occurs, DFS backtracks along the traversed edges emanating from that node until it encounters a node with an unvisited neighbor. It then proceeds along that edge and continues in this manner until all nodes within the graph have been visited. DFS can run iteratively or recursively (Tarjan).

X-CAVATE adapts DFS to identify collision-free traversal of vascular networks. The first modification is the introduction of a validity function, which determines whether, on a given DFS iteration, a node is "invalid." An invalid node within this network is one that, if printed, would result in an eventual collision between itself and the nozzle when the nozzle visits a future, as-yet unvisited coordinate. To identify invalid coordinates, X-CAVATE constructs a circular boundary region

around the node which matches the dimensions of the nozzle cross-section. It then extends this region infinitely downward in the z-direction to form a cylindrical region. If any unvisited nodes within the network fall within this cylindrical boundary, the current node is marked "invalid." The encountering of an invalid node concludes the traversal pathway of the current DFS iteration, completing the "print pass," and triggers a new iteration. This leads to the second DFS modification: the new iteration will always begin at the positionally lowest unvisited node within the network. This imposed directionality minimizes collisions by ensuring that print pass initiation proceeds from bottom-to-top within the network. While the individual coordinates within the print pass are often entirely printed bottom-to-top, some passes contain nozzle traversals that both ascend and descend in the z-direction. X-CAVATE incorporates bottom-to-top directionality at the DFS level, rather than at the edge level, to enable the printing of these segments from top-to-bottom when permissible (i.e., when doing so does not manifest in collisions).

Each iteration of the DFS becomes an individual "print pass," or segment of a vessel which will be printed continuously. The order of nodes within an individual print pass specifies the order in which the nozzle will traverse them. G-code will dictate pressure-based ink extrusion between the nodes to produce the print pass filament. Following completion of a single print pass, G-code and Aerobasic commands will pause the pressure application, lift the nozzle vertically out of the support bath, translate the nozzle in the xy-plane to the starting (X, Y) coordinates of the next print pass, lower the nozzle to the starting node, and reapply pressure to begin extrusion for printing the subsequent print pass.

9.3 Post-processing

X-CAVATE incorporates post-processing steps to handle two features of the DFS algorithmic output. First, because the DFS algorithm incorporates backtracking, it occasionally traverses non-neighboring nodes, i.e., pathways external to the intended vascular design. So, following the DFS, X-CAVATE subdivides the output print passes into additional print passes at these specific instances. Second, although the DFS ensures traversal of all nodes within the network, it does not guarantee traversal of all edges. Non-traversed edges manifest in physical gaps within the printed ink filaments. Gaps, when they arise, occur only between the endpoint of an individual print pass and its neighbor(s). For this reason, X-CAVATE includes post-processing steps to sequentially check the first and last nodes of each print pass for unconnected neighbors.

To close the gaps, X-CAVATE first searches for disconnects between standard nodes and other nodes, excluding parent branch points. It does this by checking whether the first or last node of the current print pass has any neighbors within previously-printed passes which are not adjacent to it within the current print pass ("Condition #0"). If so, it appends that neighbor to the start or end of the current print pass, respectively. Second, X-CAVATE searches for untraversed edges between daughter and parent nodes. For any print passes with one or more daughter nodes serving as endpoints, it searches for the corresponding parent branch point within previous print passes ("Branchpoint Condition #1") and, after appending the relevant nodes, within future print passes ("Branchpoint Condition #2"). Next, X-CAVATE identifies print passes whose endpoints consist of one or more parent branch points ("Branchpoint Condition #3"). If the parent branch point does not reside next to its non-daughter neighbor in the current print pass, X-CAVATE searches for the non-daughter neighbor within the previous print passes and appends as before. Finally, X-CAVATE searches for any final disconnects remaining within the network by identifying nodes which appear only once within the set of print passes and also appear as an endpoint (excluding inlets and outlets). These disconnects typically take the form of gaps between standard-node endpoints and parent

branch points embedded within existing print passes.

9.4 Radius Computation

To incorporate the specified vessel radii, X-CAVATE continuously updates the print speed of each coordinate within the network, at constant pressure, to dynamically alter the printed filament diameter. The computation of the print speed depends on the volumetric flow rate equations presented in Section 6. For a known filament radius r and ink volumetric flow rate Q, the nozzle translation velocity v can be calculated from Equation 4.

9.5 Generation of Print Passes for Multimaterial Printing

X-CAVATE enables users to specify which of the input vessels belong to the arterial circulation and which are venous. Here, the distinction between arterial and venous is referred to as "vessel type." To translate the programmed vessel type to the 3D printed model, X-CAVATE generates print paths for multi-material printing with two distinct inks: arterial ink and venous ink. Generation of multi-material print paths occurs only if the VascularNetwork.txt file contains a fifth column specifying the vessel types. To create the multi-material print passes, X-CAVATE further subdivides the post-processed print passes by vessel type, similar to the subdivision step which follows the DFS. G-code is then used to incorporate nozzle translations between subdivided print passes to physically switch inks.

To generate the multi-material passes, X-CAVATE iterates through the print passes which were already generated for single material printing to locate positions within the pass at which the vessel type of adjacent nodes differs and subdivides the passes into separate passes to permit nozzle translations between them (for switching ink). Prior to this subdivision, however, X-CAVATE may switch the assigned vessel type of individual nodes to avoid segmenting into passes consisting of single nodes. The cases in which these switches occur are as follows: 1) in the event that the vessel type of the first node in the pass differs from the vessel type of the subsequent nodes in that pass, 2) in the event that the vessel type of the last node in the pass differs from the vessel type of the preceding nodes in that pass, 3) in the event that the vessel type of an individual node within a pass differs from the vessel type of the two nodes immediately preceding and immediately following it, and 4) in the event that the print pass consists of only two nodes and the vessel types of these two nodes differ; in this case, X-CAVATE will reassign the vessel type of the first node to match that of the second node.

10 Module Structure

Text added by Claude

The original X-CAVATE codebase consisted of a single monolithic script (`xcavate.py`, approximately 5,568 lines) plus standalone calibration scripts. The refactored version is organized as a Python package with the following structure:

```
xcavate/
  __init__.py              # Package init
  __main__.py              # python -m xcavate entry point
  cli.py                   # Argument parser and CLI entry point
  config.py                # XcavateConfig dataclass and enums
```

```
pipeline.py              # run_xcavate() orchestrator

core/
  calibration.py          # compute_flow_rate(), validate_calibration()
  gap_closure.py          # Gap closure pipeline
  graph.py                # Graph construction and special node detection
  multimaterial.py        # Material classification and subdivision
  pathfinding.py          # DFS with KD-tree collision detection
  postprocessing.py       # Subdivision, downsampling, overlap, reordering
  preprocessing.py        # Interpolation and vessel boundary detection

io/
  reader.py               # Network and inlet/outlet file parsing
  writer.py               # Coordinate output file writing
  gcode/
    base.py               # Base G-code writer and custom code loading
    pressure.py           # Pressure printer G-code writer
    positive_ink.py       # Positive ink displacement G-code writer
    aerotech.py           # Aerotech 6-axis G-code writer

gui/
  app.py                  # Streamlit GUI application

spatial/
  (spatial indexing utilities)

viz/
  plotting.py             # Plotly 3D visualization functions
```

Text added by Claude

Each module corresponds to a logical stage of the original monolithic script:

`cli.py` Replaces the argparse block from `xcavate.py` lines 41–99 and the `command_line_writer.py` helper.

`config.py` Replaces the 30+ global variables from `xcavate.py` lines 101–171.

`pipeline.py` The orchestrator that wires all modules together, replacing the main execution flow of the original script.

`core/calibration.py` Replaces `calibration.py` and `calibration_validation.py`.

`core/pathfinding.py` Replaces the DFS implementation from the original script.

`io/gcode/` Replaces the G-code generation sections, with separate writers for each printer type.